

1. Overview

This project implements a directed graph with integer edge costs and a console-based user interface.

The application supports loading/saving graphs from files, graph editing operations, random graph generation, querying graph properties, and making/restoring a backup copy.

2. Project Structure

- src/Main.java

Program entry point. Tries to load graph.txt and starts the console menu.

- src/domain/DirectedGraph.java

Core graph model.

Stores:

- inbound: NavigableMap<Integer, SortedSet<Integer>>

- outbound: NavigableMap<Integer, SortedSet<Integer>>

- costs: NavigableMap<Edge, Integer>

- vertices, edges counters

- src/domain/Edge.java

Immutable edge key (start, end), comparable for sorted map storage.

- src/service/GraphIO.java

File input/output utilities.

- readGraph(fileName): reads graph from text file

- writeGraph(fileName, graph): writes graph in the same format

- src/service/RandomGraphMaker.java

Random graph generator.

- makeRandomGraph(vertices, edges, minCost, maxCost)

- src/ui/ConsoleMenu.java

Console menu that exposes graph operations to the user.

3. DirectedGraph API

- isEdge(u, v): boolean

Checks if edge u -> v exists.

- addEdge(u, v, cost): void

Adds edge if missing; throws IllegalArgumentException if already present.

- removeEdge(u, v): void

Removes existing edge; throws IllegalArgumentException if edge is missing.

- addVertex(): int

Adds a new vertex and returns its identifier.

- removeVertex(vertex): void

Removes vertex and all incident edges.

- setEdgeCost(u, v, cost): void

Updates edge cost for an existing edge.

- getEdgeCost(u, v): int

Returns edge cost; requires edge to exist.

- copyGraph(): DirectedGraph

Returns a deep copy of the graph internals.

- parseVertices(): List<Integer>

Returns existing vertex ids in sorted order.

- parseOutboundNeighbors(vertex): List<Integer>

Returns outbound neighbors in sorted order.

- parseInboundNeighbors(vertex): List<Integer>

Returns inbound neighbors in sorted order.

- `getOutDegree(vertex)`, `getInDegree(vertex)`: int
Degree queries for a vertex.

- `getVertices()`, `getEdges()`, `getCosts()`
Access graph counters and edge-cost map.

4. File Format

Graph files are text files with this format:

- First line: "<number_of_vertices> <number_of_edges>"
- Next E lines: "<start_vertex> <end_vertex> <cost>"

Example:

```
4 5
0 1 10
0 2 4
1 3 7
2 3 1
3 0 9
```

5. Console Menu Operations

The menu supports:

1. Load graph from file
2. Save graph to file
3. Generate random graph
4. Show graph summary
5. List all vertices
6. Inspect a vertex (degrees + neighbors)
7. Check whether an edge exists
8. View an edge cost
9. Update an edge cost
10. Add a vertex
11. Remove a vertex
12. Add an edge
13. Remove an edge
14. Copy current graph to backup
15. Restore graph from backup copy
16. Print all edges
0. Exit

6. Validation and Error Handling

- Invalid numeric console input throws `IllegalArgumentException`.
- Invalid vertex ids throw `IllegalArgumentException`.
- Invalid edge access (missing edge) throws `IllegalArgumentException`.
- Adding an existing edge throws `IllegalArgumentException`.
- `readGraph` rejects empty/invalid files and truncated edge lists.
- Random generator validates negative counts, invalid cost ranges, and edge counts above $\text{vertices} * \text{vertices}$.
- Restore backup operation fails if no backup was created yet.

7. Running the Application

From `javaProject` directory:

```
mkdir -p out
javac -d out $(find src -name "*.java")
java -cp out Main
```

At startup, `Main` tries to load `graph.txt`. If loading fails, the menu starts with an empty graph.

8. Notes

- Self-loops are allowed because random generation and edge insertion accept $u == v$.
- Vertices are identified by integers.
- Vertex IDs are not reindexed after deletion; `addVertex` uses $\max(\text{existing}) + 1$.